GIT SERVER — SELF-HOSTED REPOSITORY OVER SSH

Group

23127177 - Le Nhat Duy

23127200 - Nguyen Minh Kha

23127196 - Dang Quang Hung

23127463 - Nguyen Vu Minh Quang

23127448 - Luu Vinh Phat

AGENDA

1. **Introduction & context**

what the tool is and the problem it solves.

2. **Core concepts & terminology**

define the key terms a classmate needs to follow.

3. **How it works**

the main components and a simple diagram

4. **Installation & basic settings**

how to install it on Linux and the important settings.

5. **When to use & alternative**

a short, fair comparison

6. **Conclusion & Q&A**

summary, common mistakes, questions

INTRODUCTION

A **Git Server** is a centralized repository that stores Git projects and allows multiple developers to collaborate using **Git commands over SSH**. Unlike **GitHub** or **GitLab**, a self-hosted Git server gives organizations **full control** over their **source code, access permissions, and infrastructure**.

CONTEXT

Provides a central repository for sharing and synchronizing code.

Enables secure collaboration through SSH key authentication.

Gives organizations **full ownership** of their source code and data

Suitable for **private networks**, **strict security** and **compliance requirements**.

CORE CONCEPTS & TERMINOLOGY

Bare Repository

- A Git repository containing only Git data (commits, branches, tags, and metadata) **without a working directory**.
- Acts as the **central repository** on the server for collaboration.

SSH Key Authentication

- Uses a **public/private key pair** to verify a user's identity instead of a password.
- The **public key** is stored on the server, while the **private key** remains on the client.

CORE CONCEPTS & TERMINOLOGY

Clone

- Creates a complete local copy of a remote repository, including its history and branches.
- Command: `git clone <repository-url>`

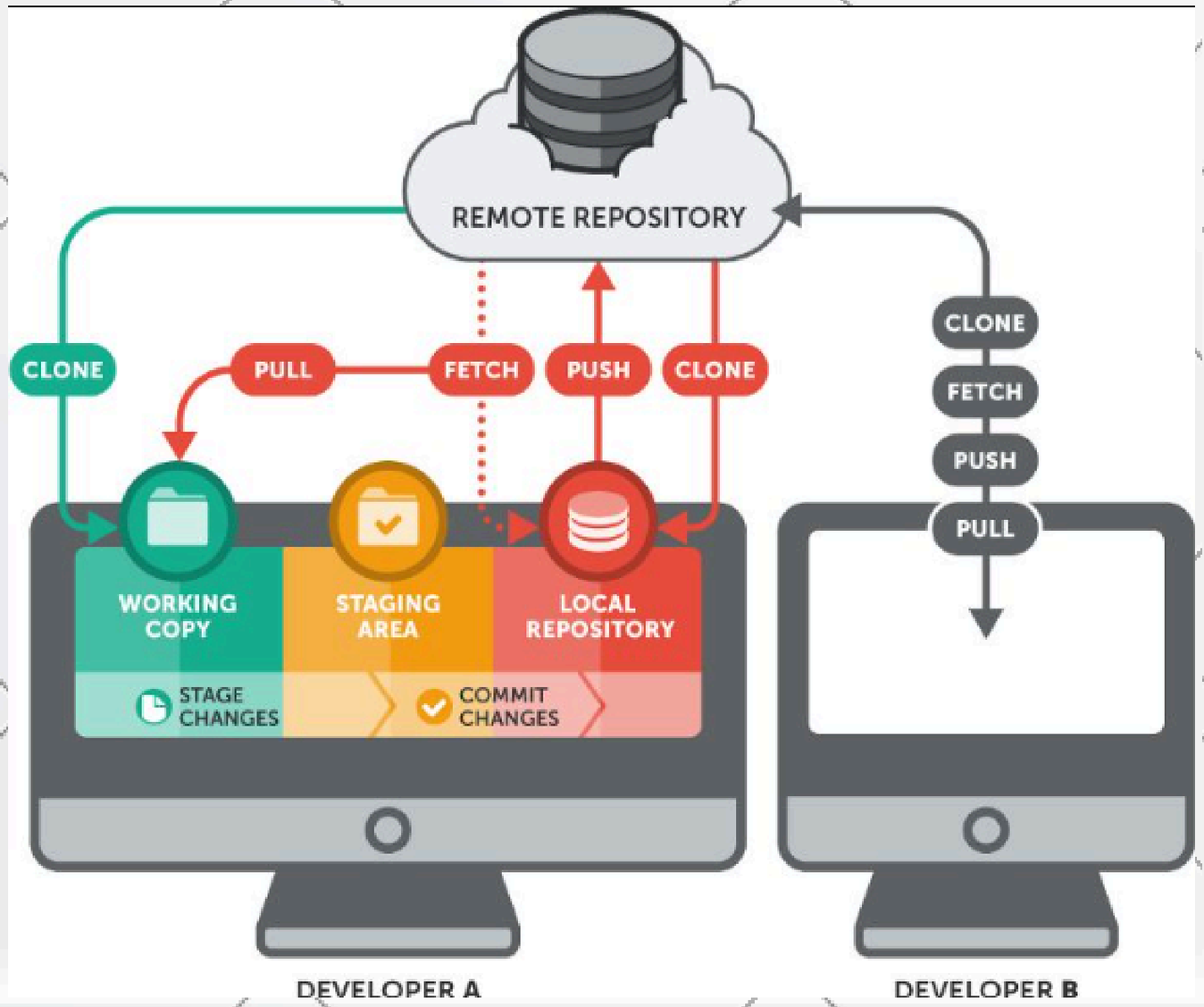
Push

- Uploads local commits from the client to the remote repository.
- Command: `git push origin <branch>`

Pull

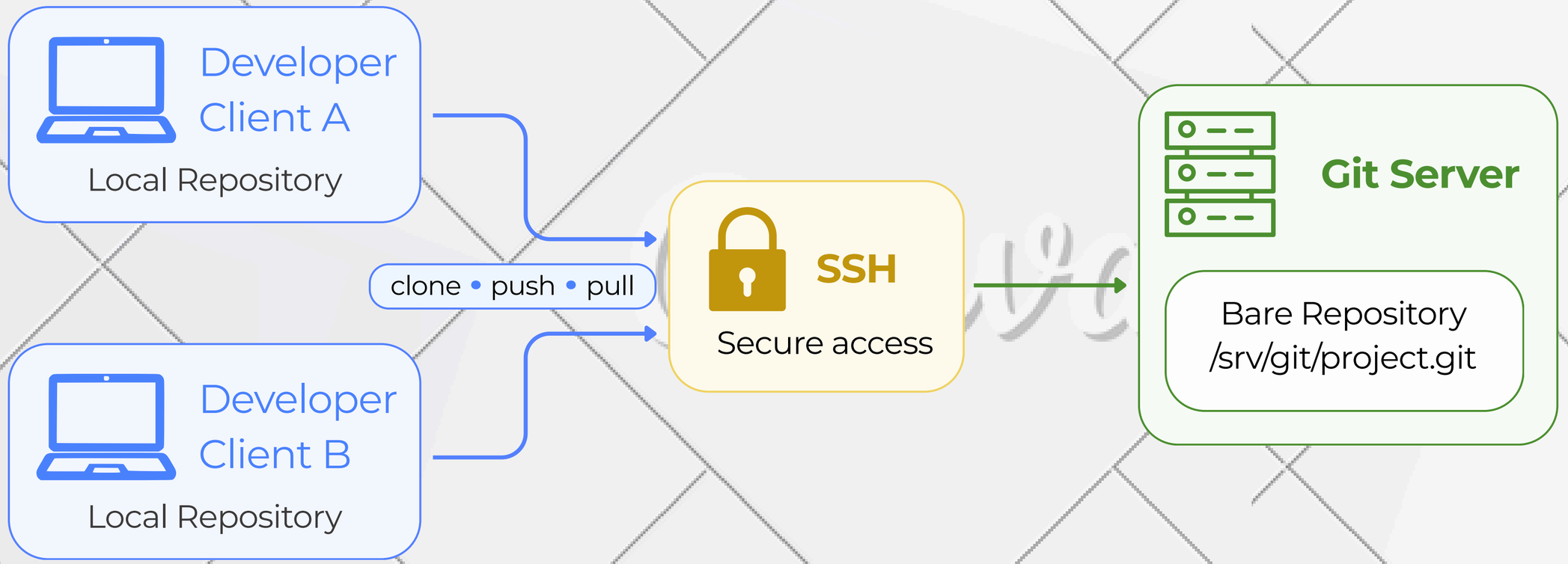
- Keeps the local repository synchronized with the server by downloading the latest commits.
- Command: `git pull origin <branch>`

CORE CONCEPTS & TERMINOLOGY



HOW IT WORKS

A Git server over SSH connects local repositories to a central bare repository



Key idea: The stores Git history, developers work locally and synchronize through SSH

HOW IT WORKS

The workflow starts with a central bare repository on the server

Administrator

Create with:

```
git init --bare /srv/git/project.git
```

Central Bare Repository

project.git

/srv/git/project.git

Purpose

- Stores Git history
- No working directory
- Acts as the central repo

Key idea: The administrator prepares one shared bare repository for all developers

HOW IT WORKS

Developers connect securely to the Git server through SSH

SSH key authentication

Developer Client

Local Repository

SSH

Encrypted transport

Git Server

Authorized access

Client side

Uses private key

Connects to the server

Server side

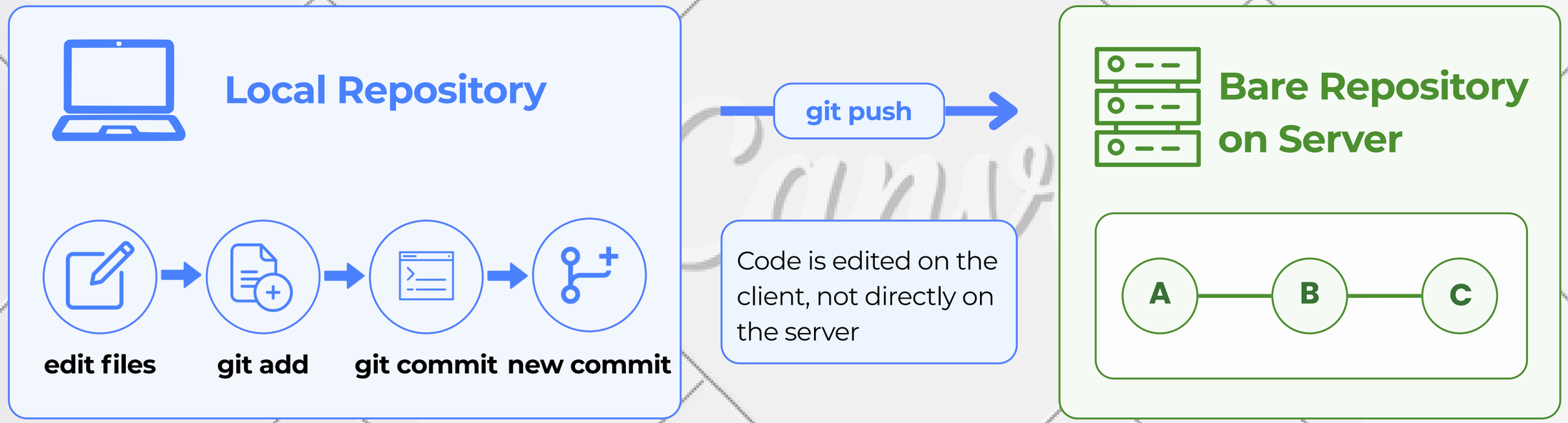
Stores public key

Checks authorized_keys

Key idea: SSH provides secure communication and verifies who is allowed to access the repository

HOW IT WORKS

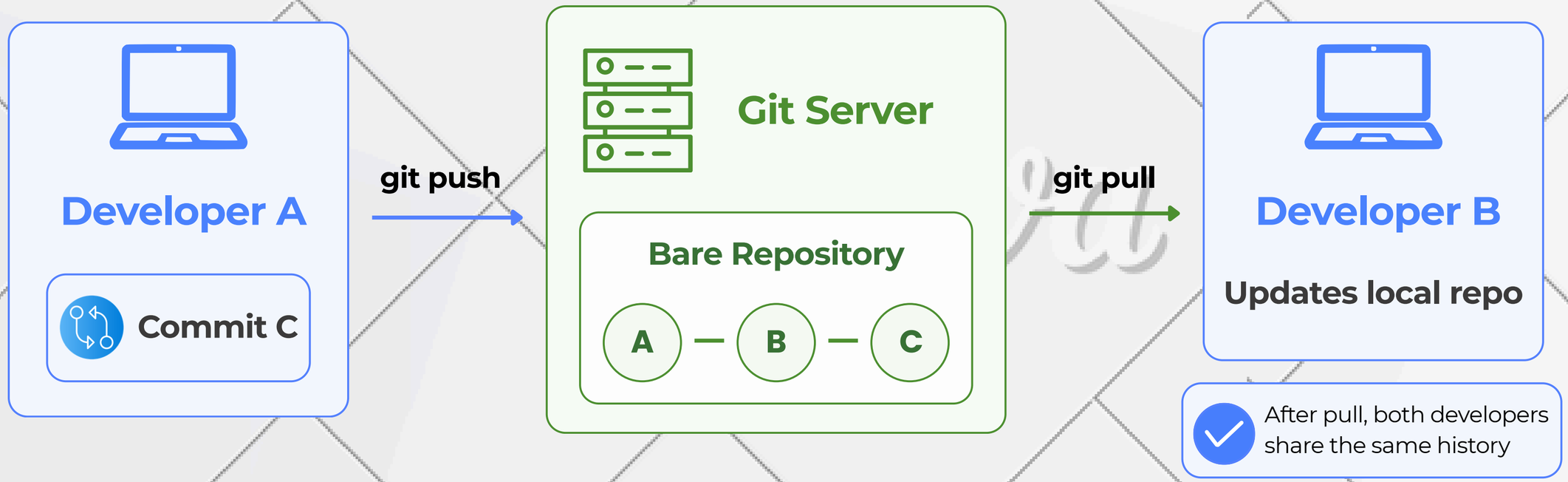
Developers work locally first then push commits to the server



Key idea: Developers create commits locally and send the new history to the central repository with push

HOW IT WORKS

The server synchronizes changes between team members



Key idea: One developer pushes changes to the server, and the others pull those changes into their local repositories.

INSTALLATION & BASIC SETTINGS

From installed packages to a verifiable, least-privilege Git service

SECURE

SSH keys, host verification, encrypted transport

TRACEABLE

One key per person or device, easy revocation

RECOVERABLE

Validate before reload
keep a rollback path

INSTALLATION & BASIC SETTINGS

A git server over SSH combines transport, indentity and repository storage

Transport

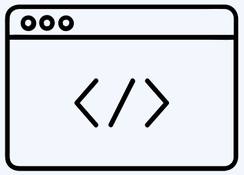
Secure chanel via SSH

Indentity

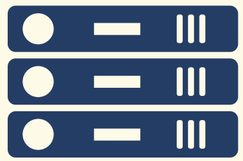
Public key proves who can connect

Repository

Bare repo stores versioned code



Developer Client



Open SSH Server

Bare Repository

INSTALLATION & BASIC SETTINGS

Install with Checkpoint

Bash

```
sudo apt update && sudo apt install -y git openssh-server
```

Bash

```
sudo systemctl enable --now ssh
```

Bash

```
sudo systemctl status ssh --no-paper -l
```

1 ACTION
Run the command

2 EVIDENCE
Check output, service, port

3 CONCLUSION
Installed and running

INSTALL CHECKLIST

Git ✓

SSH active ✓

Port 22 listening ✓

INSTALLATION & BASIC SETTINGS

Create Service Account & Install Keys

server

```
sudo adduser --disabled-password --gecos “” git
sudo install -d -m 700 -o git -g git /home/git/.ssh
sudo install -m 600 -o git -g git /dev/null
/home/git/.ssh/authorized_keys
```

server

```
sudo -u git tee -a /home/git/.ssh/authorized_keys
< /tmp/client.pub > /dev/null
```

Client

```
ssh-keygen -t ed25519 -C “client”
cat ~/.ssh/id_ed25519.pub
```

/home/git/.ssh = 700 | authorized_keys = 600 | owner: git:git

INSTALLATION & BASIC SETTINGS

Trust Works in Two Directions

Client Verifies Server

Is this the real server?

Bash

```
ssh git@server.example.com
```

know_hosts

Stores server fingerprints

Server Verifies Client

Is this the right user?

Bash

```
cat ~/.ssh/id_ed25519.pub
```

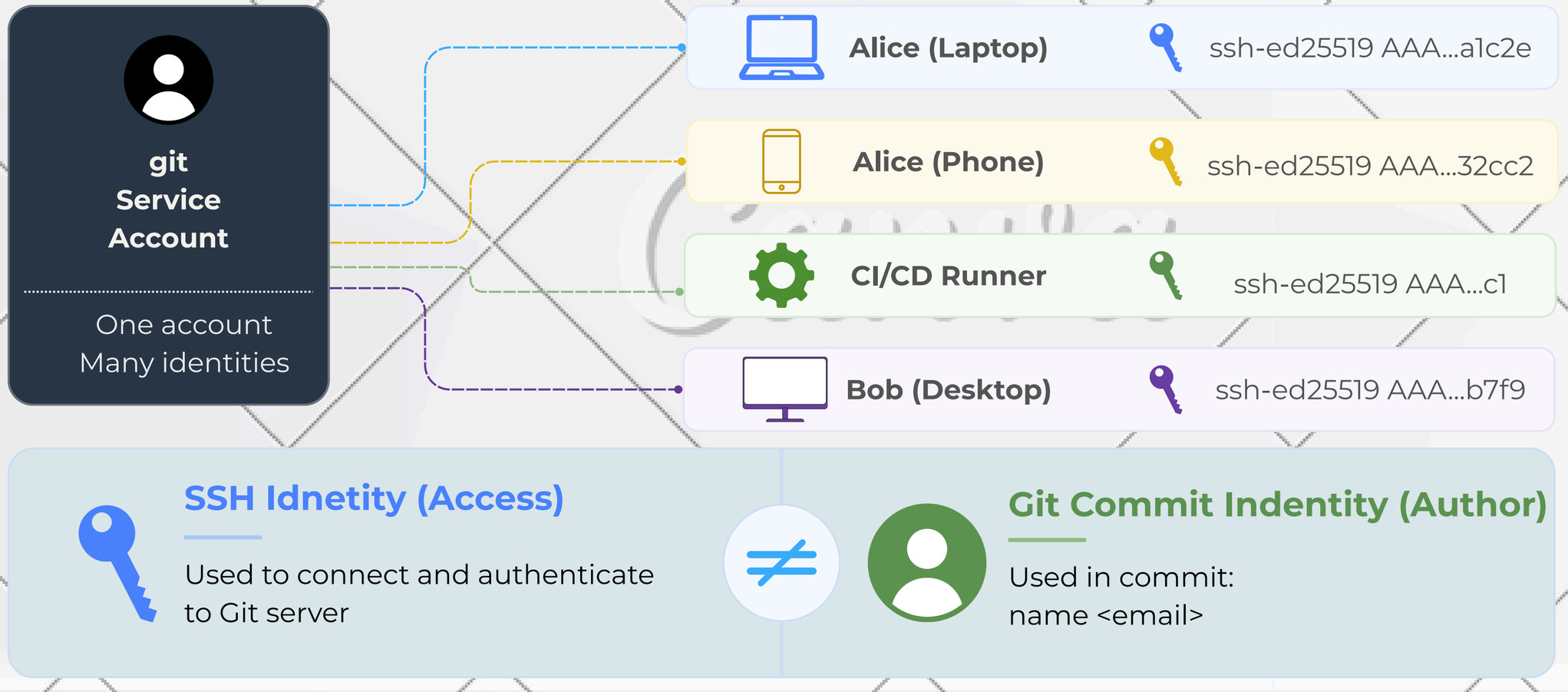
authorized_keys

Lists allowed public keys

Trust is mutal: client checks server, server checks client

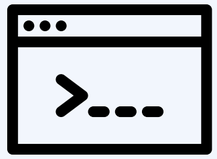
INSTALLATION & BASIC SETTINGS

One Account, Multiple Identities



INSTALLATION & BASIC SETTINGS

Least Privilege by Design



git-shell

Git commands only

Match User git

Restrictions apply only to git user

No interactive shell

No login
no forwarding

sshd_config

```
Match User git
        PasswordAuthentication no
        AllowTcpForwarding no
        PermitTTY no
        ForceCommand /usr/bin/git-shell
```

Allow only what is needed - and nothing more

WHEN TO USE IT

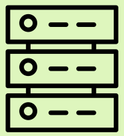
Small teams

- Ideal for teams (1–10 developers) needing core operations (Clone, Push, Pull, Merge).
- Bypasses the complexity of advanced platforms (no need for issue tracking or pull requests).

Private or Internal Projects

- Ensures strict data privacy inside an organization.
- Source code remains fully controlled by the company, not third-party cloud services.

Lightweight and Easy to Maintain



- Uses fewer system resources.
- Easy to install and configure.
- Suitable for Raspberry Pi, old servers, or virtual machines.

Lean Deployments (Automation)

- Allows the use of post-receive hooks.
- Enables instant, automated deployments to the server upon pushing code without complex CI/CD tools.

ALTERNATIVE

Self-hosted

Full control over data

SSH access only (typically)

Basic Git operations

Lightweight

No built-in automation

Cloud-hosted (by default)

Data stored on GitHub's servers

SSH or HTTPS + Web Interface

Git + Collaboration

No server maintenance, but Internet-dependent

GitHub Actions

CONCLUSION & Q&A

summary

common mistakes

question

THE END

THANKS FOR LISTENING